ENTERPRISE ARCHITECTURE V5

Professional Portfolio
2026 Manufacturing DX Project
Project

ESS 배터리 제조 공정 통합 관리 시스템

다단계 BOM 기반 연쇄 재고 차감 및 AI 융합(Python/RPA) 엔터프라이즈 아키텍처

DEVELOPMENT PERIOD

2026.03.11 — 2026.04 (4주)

CORE TARGET

배터리 제조 공정의 디지털 전환,
데이터 무결성 보장 및 100% 자체 AI 분석 파이프라인

PRESENTERS

팀장: 김강산
팀원: 정원준, 박주희

Table of Contents

전체 시스템 구성 및 핵심 구현 파트별 목차

01 **기획 배경 및 시스템 아키텍처**
ESS시장 분석 시스템 아키텍쳐 필요성

02 **[협업] 백엔드 크로스 체크 및 Git 전략**
무결성 보장을 위한 팀 협업 규칙

03 **[인사/보안] 동적 권한 제어 및 UI/UX**
박주희: Role 기반 동적 메뉴 및 대시보드

04 **[생산/BOM] 다단계 BOM 및 트랜잭션**
김강산: 재귀 전개 로직 및 롤백 프로세스

05 **[물류/영업] 블랙박스형 재고 수불 로그**
정원준: Append-Only 이력 및 예외 처리

06 **[AI/데이터] 온프레미스 AI 및 RPA**
정원준: OCR 파이프라인 및 자동 리포팅

07 **프로젝트 회고 및 ROI**
도입 성과, 기술적 성장 및 기대 효과 분석

역할 분담 (R&R)

Project Roles and Responsibilities

김강산
리더

생산 | 보안 | DB설계

- 다단계 BOM 연쇄 계산 알고리즘 설계 및 구현
- Spring Security 기반 정교한 권한 제어 모델링
- 완제품 생산 시 부품 자동 차감 트랜잭션 구축
- 데이터 베이스 구축

정원준
핵심 개발 (AI/Logistics)

물류 | 영업 | AI/RPA

- 입출고 재고 데이터의 원자적 검증 로직 구현
- Python 활용 PDF/IMG OCR 분석 파이프라인
- 머신러닝 기반 예측 데이터 엑셀 반환 로직
- Brity RPA 기반 이메일 자동 발송 워크플로우

박주희
핵심 개발 (HR/UI/UX)

인사 | 기초 | 프론트엔드

- 사원/거래처/품목 기초 데이터 CRUD 구현
- V5 동적 권한(기본 권한 보호) 로직 설계
- JSTL/Vanilla JS 기반 동적 사이드바 렌더링
- 안전재고/납기지연 대시보드 위젯 UI 연동
- Java-Python 연동 서버 아키텍처 총괄 설계

기획 배경: ESS 배터리 시장의 페인 포인트

시장 급성장에 따른 관리 복잡도 증가와 기존 방식의 한계 분석

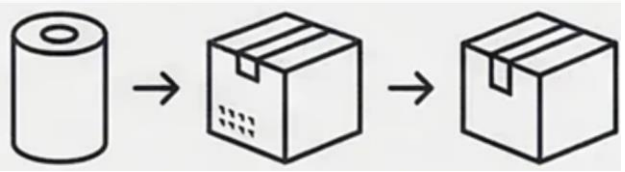
시장 폭발적 성장

- 전기차(EV) 및 신재생에너지 인프라 확대로 인한 ESS 수요 급증
- 에너지 저장 장치 시장의 글로벌 스케일업 가속화

기존 관리의 한계

CELL → MODULE → PACK

- 복잡한 다단계 조립 공정 내 엑셀 수기 관리의 한계
- 1차원적 ERP 시스템 사용으로 부품 재고 오차율 심각



솔루션 필요성

- 휴먼 에러를 원천 차단하는 **'무결성 백엔드'** 구축
- 외부 유출 없는 **'온프레미스 AI'** 결합형 ERP
- 지능형 조립 공정 맞춤형 디지털 전환 실현

ESS 배터리 제조 특화 지능형 ERP 구축 필요

개발 목표

ESS 시스템의 신뢰성과 효율을 위한 3대 핵심 개발 목표

원자적 정확성

ATOMICITY

- 다단계 BOM 레시피 기반의 원자재 자동 연쇄 차감
- 완벽한 DB 트랜잭션 설계를 통한 롤백(Rollback) 보장

추적 투명성

TRANSPARENCY

- 조작 및 덮어쓰기가 불가능한 Append-Only 로그 설계
- 블랙박스형 재고 수불 로그(TB_ITEM_LOG) 아키텍처

보안 및 자동화

SECURITY & AUTOMATION

- 외부 API 의존 없는 자체 온프레미스 AI 분석(OCR/예측)
- Brity RPA를 통한 재무 리포팅 완전 자동화 워크플로우

3가지 목표를 달성하여 데이터 무결성 ERP를 구현

#Enterprise_Standard #Data_Integrity #Automation_First

사용 기술 스택 (Tech Stack)

안정적인 제조 공정 관리를 위한 엔터프라이즈 폴리글랏 기술 구성

Backend (Core)

Java 17 | Spring Boot

강력한 트랜잭션 관리와 보안 통제를 위한 표준 프레임워크 기반 설계

Backend (AI/Data)

Python | Pandas | Statsmodels

온프레미스 기반 OCR 처리 및 시계열 매출 예측 알고리즘 구현

Database

MariaDB | MyBatis

3정규화를 준수한 RDBMS 설계 및 SQL 중심의 유연한 데이터 매핑

Frontend & UI

JSP / JSTL | Bootstrap | Vanilla JS

서버 사이드 렌더링 기반의 동적 메뉴 구성 및 직관적인 대시보드 UX

Automation

Brity RPA

분석 결과 자동 다운로드 및 이메일 발송을 통한 무인 리포팅 환경 구축

System Integration

Java 서버와 Python AI 서버 간의 비동기 통신 및 RPA 연동을 통해 수기 작업 99% 단축 목표 달성

ENTERPRISE READY FULL-STACK SECURITY AI & RPA INTEGRATED

폴리글랏(Polyglot) 시스템 아키텍처

Java 코어 서버와 Python AI 서버의 상호운용성 및 MSA 지향 설계

FRONTEND LAYER
JSP / JSTL / Bootstrap 5

Core Java Server

- Spring Boot 3.x MVC (Controller-Service-Mapper)
- Spring Security 기반 전역 보안 및 권한 통제
- BOM 연쇄 계산 및 원자적 트랜잭션 관리

AI Data Server

- Python FastAPI 기반 비동기 분석 엔진
- Statsmodels / Scikit-Learn 시계열 예측
- OCR 텍스트 파싱 및 Excel 리포트 생성

MariaDB (Central RDBMS)

Brity RPA (Automation Pipeline)

Key Flow

데이터 무결성을 보장하는 Java MVC 구조와 고성능 분석을 수행하는 Python MSA의 유기적 결합

#REST_API #Asynchronous #On_Premise_AI

데이터베이스 아키텍처 (총 18개 테이블 정규화)

데이터 중복 최소화(3NF) 및 외래키(FK) 무결성 기반의 전사적 데이터 모델링

3정규화(3NF) 준수 | FK 제약조건 무결성 | Append-Only 로그 설계

8 TABLES

권한/인사 파트

TB_DEPT
TB_ROLE
TB_EMP_ROLE
TB_EMP
TB_MENU
...

다대다 매핑 및 동적 권한 제어를 위한 정규화 구조

6 TABLES

생산/영업 파트

TB_BOM
TB_WORK_ORDER
TB_ORDER
TB_WORK_PERF
...

BOM 재귀 참조 및 공정 실적 추적을 위한 연계 설계

3 TABLES

재고 파트

TB_ITEM
TB_ITEM_LOG
TB_WAREHOUSE

마스터(현재고)와 **이력(로그)**의 물리적 분리를 통한 무결성 확보

1 TABLE

AI 파트

TB_RECEIPT

OCR 추출 데이터 및 분석용 비정형 텍스트 수집 적재

백엔드 팀 협업: 3대 크로스 체크 룰

데이터 무결성과 시스템 안정성을 위한 백엔드 리더 주도의 기술 전략

공통 로직 모듈화

CommonItemService 강제

재고 증감(IN/OUT)과 이력(Log) 저장은 무조건 **CommonItemService** 단일 모듈을 거치도록 강제하여 비즈니스 로직의 파편화를 방지합니다.

전역 작업자 추적

SecurityContextHolder

로그인한 사번(EMP_ID)을 파라미터가 아닌 **전역 컨텍스트**에서 추출하여 모든 로그에 자동 바인딩함으로써 데이터 조작 방지 및 책임 추적성을 확보합니다.

FK 데이터 무결성

더미 데이터 SQL 공유

외래키(FK) 제약조건 에러를 방지하기 위해 조장 주도의 **기초 데이터 INSERT SQL**을 공유하여 전 팀원이 동일한 DB 환경에서 정합성을 유지합니다.

핵심 가치: *#Atomicity #Traceability #Consistency*

[발표: 박주희 - 인사/보안/프론트엔드] 파트 소개 및 목표

PRESENTER
박주희

담당 영역 (Responsibilities)

기초 데이터 CRUD
사원, 거래처, 품목 정보의 안정적 관리 모듈 구현

V5 동적 권한 제어 모델링
부서/개인별 유연한 권한 할당 및 보안 로직 설계

메인 대시보드 UI/UX 연동
사용자 중심의 직관적인 데이터 시각화 위젯 개발

핵심 목표 (Core Objectives)

- 데이터 오타를 방지하는 꼼꼼한 **유효성 검사**
- 부서 이동 시 즉각 반영되는 유연한 **엔터프라이즈 권한 제어**
- 업무 효율을 극대화하는 **직관적 UX 제공**

"시스템의 첫인상과 보안의 기초를 설계하여, 사용자가 가장 신뢰할 수 있는 인터페이스를 구축합니다."

[인사/보안] V5 사원-권한 동적 제어 시스템

사원별 맞춤 권한 부여 및 시스템 핵심 권한 보호 로직 설계

설계 규칙 (TB_EMP_ROLE)

다대다(N:M) 매핑 테이블에 **IS_DEFAULT** 속성을 도입하여 권한의 출처와 중요도를 구분하여 관리합니다.

Y **기본 권한 (Default Role)**

사원 등록 또는 부서 이동 시 시스템이 자동으로 부여하는 부서별 필수 권한 세트

N **추가 권한 (Manual Role)**

특정 업무 수행을 위해 관리자가 수동으로 개별 부여한 커스텀 권한

설계 의도

사원의 부서가 변경되더라도 수동으로 부여받은 특수 권한(N)은 유지하면서, 기존 부서의 기본 권한(Y)만 새로운 부서의 것으로 동적 교체할 수 있는 유연한 아키텍처를 지향함.

핵심 방어 로직 (DB Level)

관리자가 권한 해제 시, 실수로 부서의 '기본 권한'을 삭제하지 못하도록 Mapper 레벨에서 강제 조건을 부여합니다.

```
-- EmpRoleMapper.xml: 권한 삭제 쿼리

DELETE FROM TB_EMP_ROLE
WHERE EMP_ID = #{empId}
  AND ROLE_CD = #{roleCd}

-- [원천 차단] 기본 권한은 삭제 프로세스에서 제외
  AND IS_DEFAULT = 'N'
```

EXPECTED RESULT

- ✅ 수동 권한('N') 요청 시에만 행 삭제 성공
- ❌ 기본 권한('Y') 요청 시 0건 삭제 (무시됨)

[인사/보안] RoleMapper: 권한 보유 여부 즉시 판별

DB 레벨의 최적화된 쿼리를 통한 UI 렌더링 효율성 극대화

최적화 배경 및 목표

권한 관리자 화면에서 사원의 권한을 체크박스로 직관적으로 보여주기 위해, 애플리케이션 단의 반복문이 아닌 **단일 SQL 쿼리**로 데이터 정제 수행.

LEFT JOIN 전략
전체 권한 테이블(tb_role) 기준 사원별 매핑 테이블(tb_emp_role) 결합

IF문 실시간 판별
EMP_ID 존재 여부에 따라 'Y', 'N'을 DB에서 즉시 계산하여 전송

고효율 아키텍처
백엔드 연산 부하 감소 및 프론트엔드 체크박스 바인딩 속도 향상

SQL - ROLEMAPPER.XML

```xml
<!-- 역할 전체 목록 조회 (사원 보유 여부 포함) -->
<select id="selectEmpRoleStatus" parameterType="String" resultType="map">
    SELECT
        tb_role.ROLE_CD,
        tb_role.ROLE_NM,
        tb_emp_role.IS_DEFAULT,
        IF(tb_emp_role.EMP_ID IS NOT NULL, 'Y', 'N') AS HAS_ROLE
    FROM tb_role
    LEFT JOIN TB_EMP_ROLE
        ON tb_role.ROLE_CD = tb_emp_role.ROLE_CD
        AND tb_emp_role.EMP_ID = #{empId}
</select>
```

Point: JOIN 조건에 를 직접 명시하여 특정 사원의 권한 보유 여부만 정확히 필터링합니다.

[인사/보안] JSTL 기반 동적 메뉴 및 대시보드 위젯 렌더링

JSTL 기반 동적 사이드바

Security Session의 **TB_ROLE_MENU** 권한 리스트 연동

JSP <c:forEach>를 활용한 권한별 메뉴 필터링

Accordion 스타일의 하위 메뉴 동적 렌더링

비인가 메뉴 접근 시 프론트엔드 레벨 1차 차단

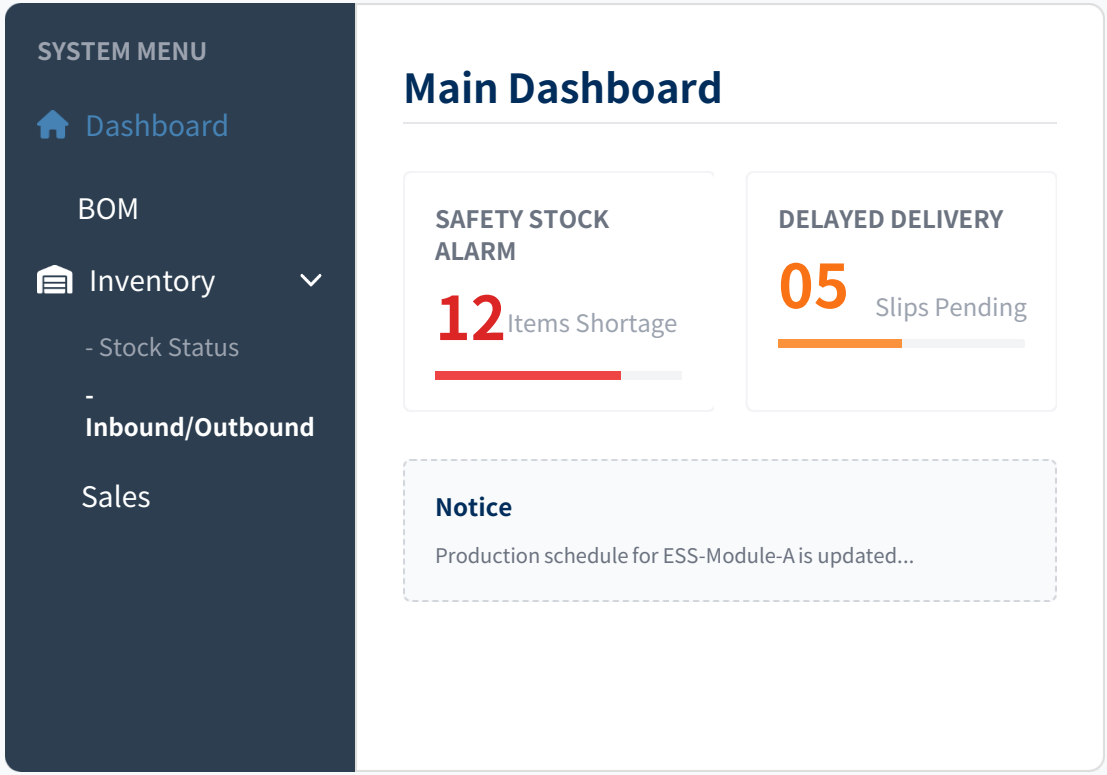
스마트 업무 위젯 (UX)

dashboard.jsp 메인 화면 실시간 데이터 바인딩

안전재고 미달 건수: 자재 부족 리스크 즉시 인지

납기 지연 전표: 영업/물류 지연 상황 우선 파악

로그인 직후 별도 검색 없이 핵심 지표 노출



JSP / JSTL | Spring Security Session | Vanilla JS

terface

[인사/보안] Spring Security & BCrypt 단방향 암호화

데이터 무결성 및 보안 내재화를 위한 계정 보호 아키텍처

보안 강화 목적

- 사원 비밀번호 탈취 및 **내부자 위협 방지**
- DB 내 **평문(Plain Text) 저장 원천 차단**
- 시스템 관리자조차 **복호화 불가능**

BCrypt 알고리즘 특성

솔팅(Salting)과 키 스트레칭(Key Stretching)을 지원하여
무작위 대입 공격(Brute-force)에 강력한 저항력을 가짐.

STEP 01. INPUT

초기 비밀번호 입력 (Plain Text)

"mypassword123!"

STEP 02. PROCESS

BCryptPassword.encode()

단방향 해시 알고리즘 적용 (Salting 포함)

STEP 03. STORAGE

60자 Hash 문자열 DB 저장

복호화 불가

"$2a$10$vI8...[60 characters]...X8vG"

TECH STACK Spring Security 6.x

ENCRYPTION BCrypt Hash

PART 04. PRODUCTION & BOM

[발표: 김강산 - 생산/BOM] 파트 소개 및 목표

김강산

총괄 리더 (Total Leader)

담당 영역

다단계 BOM 명세서 관리

핵심 기능

작업 지시 및 생산 실적 트랜잭션

주요 기술 스택

Spring Boot | MyBatis | Spring Security | MariaDB

핵심 목표 (Core Mission)

*"완제품을 생산할 때 수십 개의 원자재가 정확한 비율로 **줄줄이 빠져나가는 연쇄 계산 로직(Explosion)** 구현과 **트랜잭션 롤백** 보장"*

KEYWORD 01

재귀적 BOM 전개

KEYWORD 02

원자적 트랜잭션

Integrity Automation Precision

[생산/BOM] 다단계 BOM (Bill of Materials) 모델링

완제품-부품 간 계층 구조 정의 및 고효율 데이터 추출 아키텍처

BOM 데이터 모델링 전략

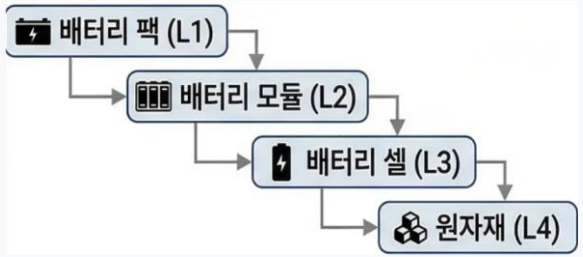
01 **트리(Tree) 구조 레시피:** 완제품 1개 생산을 위한 하위 부품 소요량 (Req Qty)을 정의하는 제조 핵심 데이터.

02 **자기 참조형(Self-Referencing) 설계:** 동일한 품목 테이블 (TB_ITEM) 내에서 상위(Parent)와 하위(Child) 관계를 매핑.

03 **고난도 쿼리 적용:** 상위/하위 품목명을 동시에 가져오기 위해 TB_ITEM을 이중 LEFT JOIN 처리하여 조회 성능 최적화.

SQL Logic Self-Join Query

```sql
SELECT
    P.ITEM_NM AS PARENT_NM,
    C.ITEM_NM AS CHILD_NM,
    B.REQ_QTY
FROM
    TB_BOM B
LEFT JOIN
    TB_ITEM P ON B.PARENT_CD = P.ITEM_CD
LEFT JOIN
    TB_ITEM C ON B.CHILD_CD = C.ITEM_CD
```



ESS 배터리 팩 BOM 계층 구조 예시

FINISHED PRODUCT
ESS 배터리 팩
1 Unit

Sub-Assembly
배터리 모듈 A
Qty: 6

Material
팩 케이스
Qty: 1

배터리 셀
Qty: 10

BMS 장치
Qty: 1

완제품 생산 시 하위 부품들의 소요량이 재귀적으로 계산되어 재고에서 자동 차감됩니다.

데이터 무결성 검증 완료

실시간 연쇄 계산 엔진 연동

[Trouble Shooting] BOM 무한 루프 이중 방어 아키텍처

BOM 순환 참조 방지를 위한 프론트-백엔드 교차 검증 시스템

⚠ 치명적 위험: 순환 참조 (Circular Reference)

상위 품목과 하위 품목을 동일하게 등록할 경우, 생산 실적 처리 시 **재귀적 연쇄 차감 로직**이 무한 루프에 빠져 서버 리소스가 고갈되고 데이터 정합성이 훼손됨.

1 Front-end: 1차 방어 (JS)

Vanilla JS를 활용하여 폼 제출 전 여부를 즉시 판단하여 사용자에게 경고 후 차단.

2 Back-end: 2차 방어 (DB)

클라이언트 검증 우회 공격에 대비하여 Service 레이어에서 쿼리를 통해 DB 레벨의 중복 조합 검증 수행.

```java
// BomService.java
// 자기 참조 방어
if (bomDTO.getParentCd().equals(bomDTO.getChildCd())) {
    throw new IllegalArgumentException(
        "상위 품목과 하위 품목은 같을 수 없습니다."
    );
}

// 중복 BOM 등록 방어
int dupCount = bomMapper.checkDuplicateBom(bomDTO);
if (dupCount > 0) {
    throw new IllegalArgumentException(
        "이미 등록된 BOM 조합입니다."
    );
}
```

방어 프로세스 파이프라인

사용자 입력 (Parent = Child)

↓

Front-end JS 1차 차단
e.preventDefault() / Alert — STOP

↓

Back-end DB 레벨 2차 차단
Validation Fail / 롤백 실행 — STOP

↓

무결성 보장 & 무한 루프 원천 차단

PRODUCTION MANAGEMENT

[생산/BOM] 실시간 생산 실적 트랜잭션 파이프라인

작업자가 '양품 수량'을 기입하는 순간, 백엔드에서 0.1초 만에 수행되는 원자적 프로세스

01

실적 데이터 생성

TB_WORK_PERF 테이블에 생산 실적 데이터 INSERT

02

BOM 레시피 색인

완제품 기준 하위 부품(BOM) 목록 및 소요량 SELECT

03

[핵심] 부품 차감

원자재 재고 출고(-) 처리 및 수불 로그(LOG) 반복 기록

04

완제품 입고

생산된 완제품 재고 가산(+) 및 수불 로그 기록

05

지시 상태 갱신

작업지시(WORK_ORDER) 상태를 'DONE'으로 변경

Spring @Transactional Scope

단 하나라도 실패 시 전체 Rollback (원자성 보장)

SERVICE WorkPerfService.java MAPPER WorkPerfMapper.xml

Tech Highlight: BOM 연쇄 차감 알고리즘

핵심 비즈니스 로직

ERP 백엔드의 꽃이자 가장 기술적 난이도가 높았던 부분입니다. 완제품 생산 시 **수십 개의 원자재**가 정확한 비율로 줄줄이 빠져나가는 연쇄 계산을 수행합니다.

구현 포인트

- 생산 완제품 수량에 맞춰 하위 부품들의 1개당 소요량(reqQty)을 재귀적/반복적으로 계산
- CommonItemService를 통한 일괄 재고 소모(OUT) 및 실시간 로그 기록
- 데이터 무결성을 위한 원자적 처리(Atomic Processing)

WorkPerfService.java

```java
// 4. [핵심] 원자재 연쇄 차감 (원자재 소모량 = BOM 소요량 * 생산된 양품 수량)
if (children != null) {
    for(BomDTO child : children) {
        int deductQty = child.getReqQty() * perfDTO.getGoodQty();

        // [추가] 원자재 재고 검증 로직 (마이너스 재고 방지)
        int currentStock = commonItemMapper.selectCurrentStock(child.getChildCd());
        if (currentStock < deductQty) {
            throw new RuntimeException(
                "원자재 재고가 부족하여 생산을 진행할 수 없습니다. (필요 부품: " + child.getChildCd() +
                ", 현재고: " + currentStock + "개, 필요수량: " + deductQty + "개)");
        }

        // 공통 서비스를 활용해 재고 마이너스 처리 및 OUT 이력 기록
        commonItemService.updateStockAndLog
        (child.getChildCd(), -deductQty, inoutType: "OUT", perfDTO.getWorkNo(), perfDTO.getEmpId());
    }
}
```

```
        );
    }
}
```

[Trouble Shooting] @Transactional 기반 롤백 보장

Production/BOM Logic Integrity

⚠ 문제 인식: 데이터 오염 위험

10개의 하위 부품 중 9개는 재고가 차감되었으나, 마지막 1개가 창고 재고 부족으로 에러가 발생할 경우 **DB 데이터 불일치(Inconsistency)**가 심각하게 발생함.

✔ 해결책: 원자적 트랜잭션 설계

Spring **@Transactional** 어노테이션 부여

반복문 내 **실시간 재고 검증 로직** 삽입

재고 부족 시 명시적 **RuntimeException** 발생

전체 생산 내역을 이전 상태로 **강제 Rollback**

ATOMIC ROLLBACK FLOW

Start › Deduct #1~9 › #10 Fail! › ROLLBACK › Safe

WorkPerfService.java — Java / Spring Boot

```java
@Transactional
public void processProduction(PerfDTO perfDTO) {// ... 중략 ...

    for (BomDTO child : children) {int currentStock =
commonItemMapper.selectCurrentStock(child.getChildCd());

        // 1. 마이너스 재고 검증
        if(currentStock < deductQty) {throw new RuntimeException(
                "원자재 재고 부족! (품목: "+ child.getChildCd() +", 현재고: " + currer
                "개, 필요수량: "+ deductQty + "개)"    );
                 commonItemService                    }
        // 2. 재고 차감 및 로그 기록 (실패 시 위 예외로 롤백)
.updateStockAndLog(...);
    }

// 3. 완제품 입고 처리    commonItemService.updateStockAndLog
}                                (parentCd, goodQty, ...);
```

** @Transactional은 RuntimeException 발생 시 자동으로 전체 DB 연산을 취소합니다.*

[발표: 정원준 - 물류/영업] 파트 소개 및 목표

물류 데이터의 무결성 확보 및 실시간 수불 관리 아키텍처

정원준

물류/영업/AI 담당

- 발주(구매) 및 수주(판매) 전표 관리
- 공통 재고 증감 모듈(Common Service) 설계
- 물류 수불 이력(Log) 관리 시스템 구축

LOGISTICS MANAGEMENT SCOPE

PO/SO Slip → Inventory → Black-box Log

Core Objectives

데이터 무결성 (Integrity)

돈과 직결되는 창고의 입출고 데이터가 1원, 1개라도 틀리지 않도록 원자적 트랜잭션 보장

마이너스 재고 차단

현실적으로 불가능한 '마이너스 재고' 발생을 백엔드 비즈니스 로직 레벨에서 원천적으로 차단

블랙박스형 이력 관리

수정 및 삭제가 불가능한 Append-Only 방식의 수불 로그를 통해 모든 물류 이동의 완벽한 추적성 확보

LOGISTICS & SALES

[물류/영업] CommonItemService 단일 창구화 및 Security 연동

문제 상황

다수의 컨트롤러(발주, 수주, 생산 등)에서 개별적으로 재고 UPDATE를 시도할 경우, 비즈니스 로직 파편화 및 이력 누락 위험 발생.

아키텍처 원칙

모든 재고 증감 및 로그 저장은 **CommonItemService**로 캡슐화

단일 모듈화를 통한 유지보수성 및 데이터 무결성 확보

보안 추적 (Security Trace)

파라미터 변조 방지를 위해 SecurityContextHolder에서 전역적으로 사번을 추출하여 모든 로그에 자동 바인딩.

STOCKCONTROLLER.JAVA

BACKEND SECURITY IMPLEMENTATION

```java
// StockController.java 일부: 보안 세션 기반 사번 추출
Authentication auth =
    SecurityContextHolder.getContext().getAuthentication();

String empId = (auth != null) ? auth.getName() : null;

// 1. 클라이언트가 넘긴 사번 파라미터가 아닌
// 2. 서버 세션에서 안전하게 추출된 empId를 사용

// 비즈니스 로직 호출 (재고 출고 프로세스)
stockService.processOutbound(orderNo, empId);

// CommonItemService 내부에서 로그(Log) 생성 시
// 해당 empId가 작업자로 자동 기록됨
```

Security Context → CommonItemService

[물류/영업] 블랙박스형 재고 수불 로그 (Append-Only) 설계

데이터 무결성과 추적성을 보장하는 이원화된 데이터베이스 아키텍처

TB_ITEM (마스터)

실시간 재고 현황 조회를 위한 마스터 테이블.
성능 최적화 및 동시성 제어를 기반으로 한 현재고 관리.

INSERT (신규 품목)	ALLOW ✅
UPDATE (재고 덮어쓰기)	ALLOW ✅
DELETE (품목 삭제)	ALLOW ✅

Query: UPDATE TB_ITEM SET STOCK_QTY = STOCK_QTY + #{qty} ...

TB_ITEM_LOG (로그)

금융권 통장 방식의 'Append-Only' 블랙박스 이력.
시스템 내 모든 입출고 내역에 대한 절대적 무결성 보장.

INSERT (이력 추가)	ALLOW ✅
UPDATE (이력 수정)	DENY ❌
DELETE (이력 삭제)	DENY ❌

Constraint: 물리적 수정/삭제 쿼리 시스템 배제 (Append-Only)

설계의 핵심 가치
조작 및 덮어쓰기가 불가능한 물리적 수불 이력 추적 환경 완성

PERFORMANCE
TB_ITEM

INTEGRITY
TB_ITEM_LOG

TROUBLE SHOOTING

이중 검증을 통한 마이너스 유령 재고 역전 방어

⚠ 치명적 에러 (Critical Error)

수주(SELL) 출고 처리 시, 전산상의 창고 재고를 초과하여 출고되면 **마이너스 재고**가 발생하여 장부와 실물 간의 심각한 불일치 초래.

1 StockService 1차 검증

출고 프로세스 진입 전 DB의 을 통해 비즈니스 로직 레벨에서 수량 확인.

2 CommonItemService 2차 검증

실제 DB 반영 직전 최종 재고()를 계산하여 0 미만일 경우 예외를 던져 **전체 트랜잭션 롤백** 유도.

CommonItemService.java

```
// 2차 마이너스 재고 방어 로직
int beforeQty = commonItemMapper.selectCurrentStock(itemCd);
int afterQty = beforeQty + qty;

if (afterQty < 0) {
    throw new RuntimeException(
        "재고 부족! (품목: " + itemCd
        " 현재: " + beforeQty +
        ", 요청: " + Math.abs(qty) + ")"
    );
}
// 무결성이 확인된 경우에만 업데이트 수행
updateStock(itemCd, qty);
insertItemLog(logDTO);
commonItemMapper.
```

Atomic transaction is guaranteed by @Transactional annotation.

Core Logic Integrity Fault Tolerance

TROUBLE SHOOTING #04

예외 처리 UI/UX 파이프라인 (Exception Handling)

문제 상황 (Pain Point)

서비스 단에서 발생시킨 **RuntimeException**(재고 부족 등)이 WAS 기본 에러 페이지(500 Error)를 노출시켜 사용자 경험(UX)을 심각하게 저해함.

아키텍처 개선 (Solution)

- Controller에서 비즈니스 로직을 try-catch로 캡슐화
- 에러 메시지를 **RedirectAttributes**로 바인딩
- **FlashAttribute**를 활용해 일회성 데이터 전달
- JSP 프론트엔드에서 팝업(Alert/Toast) 자동 렌더링

풀스택 예외 처리 흐름도

Service 예외 발생 → Controller try-catch → Flash Attribute → JSP 메시지 수신 → 사용자 알림 팝업

```
try {
        stockService.processOutbound(orderNo, empId);
  }
  catch (RuntimeException e) {
          rttr.addFlashAttribute("errorMsg", e.getMessage());
    return "redirect:/stock/list";
                                  }
```

[AI/데이터] 파트 소개 및 온프레미스 AI 도입 비전

보안 무결성과 자동화 가치를 실현하는 자체 분석 파이프라인

PRESENTER
정원준 ,박주희

기존 시스템의 한계

- 외부 AI API 사용 시 기업 핵심 재무/생산 데이터 외부 유출 위협
- 인터넷 의존적 시스템으로 인한 네트워크 장애 시 분석 중단 리스크

AI 파트 핵심 비전

"외부 의존성 0% (Zero-Dependency)"

자체 구축 Python 서버 내에서 OCR 분석부터 머신러닝 예측까지 수행하는 폐쇄형 파이프라인을 구축하여 보안과 성능을 동시에 확보합니다.

On-premise Security
데이터 유출 원천 차단, 내부 서버 처리

OCR & ML Pipeline
비정형 전표 인식 및 시계열 매출 예측

RPA Integration
분석 결과 엑셀 반환 및 이메일 자동화

AI DATA FLOW ARCHITECTURE

DB Data > Python AI > Excel Result > RPA Mail

Confidential & Internal Only

[AI/데이터] test_analytics.py: 2년 치 ERP 시계열 데이터 시뮬레이션

시뮬레이션 개발 목적

신규 시스템 도입 초기 **AI 학습용 백데이터 부족** 문제 해결

다양한 예외 상황(불량 발생, 재고 부족)에 대한 모델 강건성 확보

실제 운영 환경과 유사한 **730일(2년) 분량**의 시계열 데이터 구축

핵심 시뮬레이션 로직

BOM 레시피 준수
완제품 생산 시 정의된 하위 부품 비율에 따른 자동 재고 차감 시뮬레이션

현실적 변수 반영 (2~3% 불량률)
생산량 대비 일정 비율의 불량(bad_qty)을 랜덤하게 발생시켜 데이터 편향 방지

```
# test_analytics.py pseudo-logic
for day in range(730):
  prod_qty = random.normal(avg_qty, std_dev)
  bad_qty = prod_qty * random.uniform(0.02, 0.03)
  good_qty = prod_qty - bad_qty
  # BOM 기반 원자재 차감 SQL 생성
  generate_sql(good_qty, bom_structure)
>>> 730 days of SQL scripts generated.
```

시뮬레이션 결과물

730일 데이터 기간 (2년)	**2~3%** 평균 불량률 변수
100% BOM 규칙 일치	**SQL** 자동 생성 포맷

AI & DATA AUTOMATION

[AI/데이터] Brity RPA 연동: 재무/예측 엑셀 파일 메일 자동 발송

수작업 완전 배제

- 담당자의 웹 시스템 접속 및 파일 다운로드 수기 행위 **완전 폐기**
- 데이터 타건 및 이메일 전송 프로세스의 **자동화 전환**
- 보안이 강화된 **Unattended(무인)** 운영 환경 구축

기대 효과

업무 시간
99% 단축

데이터 정확도
100% 보장

RPA Pipeline Workflow

1. **스케줄링 및 시스템 접근**
   사전에 지정된 시간에 맞춰 프론트(웹) 시스템 자동 로그인 및 접근
2. **예측 결과 엑셀 자동 다운로드**
   Python AI 서버가 생성한 분석/예측 엑셀 파일을 웹 인터페이스를 통해 수집
3. **데이터 정제 및 서식화**
   다운로드된 로우 데이터를 경영진 보고용 양식으로 재정의 및 폰트/서식 적용
4. **유관 부서 메일 자동 발송**
   경영진 및 생산/재무 부서로 리포팅 파일을 첨부하여 무인(Unattended) 메일 발송

[AI/데이터] Python 기반 PDF/IMG 파일 OCR 파싱 및 데이터 보정

비정형 데이터 추출 (OCR)

외부에서 유입되는 영수증, PDF, 이미지 내의 텍스트를 Python 비전 라이브러리를 통해 정밀 추출합니다. 휴먼 에러가 포함된 비정형 텍스트를 디지털 데이터로 전환하는 첫 번째 단계입니다.

Fuzzy Matching 알고리즘

추출된 텍스트와 DB 표준 품목명 간의 유사도를 계산합니다. **difflib.get_close_matches**를 활용하여 오타가 포함된 단어를 표준 코드로 자동 맵핑합니다.

- ✓ 임계치: 유사도 60% 이상 시 자동 보정
- ✓ 예: "수소셀" → "H-CELL-100" 자동 전환

DATA PROCESSING PIPELINE

PDF/IMG Upload > OCR Extraction > Fuzzy Matching > TB_RECEIPT DB

fuzzy_correction.py

Similarity Threshold: 0.6

```python
import difflib

# DB에 등록된 표준 품목 리스트
standard_items = ['H-CELL-100', 'BMS-MAIN-V2', 'PACK-CASE-S']

def correct_item_name(ocr_text):
    # 유사도 60% 이상의 가장 근접한 단어 1개 추출
    matches = difflib.get_close_matches(ocr_text, standard_items, n=1, cutoff=0.6)

    if matches:
        return matches[0] # 보정된 표준 코드 반환
    else:
        return None # 매칭 실패 시 수동 검토 대상

# 실행 예시: "수소셀" 입력 시 "H-CELL-100" 매핑
result = correct_item_name("수소셀")
print(f"Corrected: {result}")
```

기술적 가치: 비정형 문서 데이터의 자동 보정을 통해 데이터 진입 단계의 무결성을 95% 이상 확보하였습니다.

Presenter: 정원준

[AI/데이터] DB 일괄 수신 및 분석 결과 엑셀(Excel) 자동 생성

Python 기반 데이터 분석 및 리포팅 자동화 파이프라인

파이프라인 통신

백엔드 DB에 적재된 대량의 수불/생산 데이터를 Python FastAPI 서버가 일괄 수신. **Pandas DataFrame**으로 즉시 변환하여 고속 분석 수행.

재무 예측 및 수율 계산

양품(Good) vs 불량(Bad) 비율 정밀 분석

수율(Yield Rate) 자동 도출

불량 발생에 따른 **손실 비용(Loss Cost)** 역산

엑셀(Excel) 렌더링

JSON 형태가 아닌 가공된 **Excel 파일 객체**로 빌드. 웹 화면 다운로드 및 RPA 자동 발송의 핵심 소스로 활용.

DATA FLOW ARCHITECTURE

MariaDB Data Extraction
Raw Production & Inventory Data

Pandas Analytics Engine
Yield Calculation & Financial Modeling

Excel Report Generation
Automated Financial & Performance Report

WEB UI DOWNLOAD

RPA AUTO MAIL

** 외부 API 의존 없이 온프레미스 환경에서 100% 자체 구동*

Confidential & Proprietary

[AI/데이터] 데이터 전처리(결측치 제어) 및 분기별 추이 분석

데이터 무결성 확보를 위한 결측치 처리 및 분석용 데이터 정규화 가공

데이터 무결성 확보 (Cleaning)

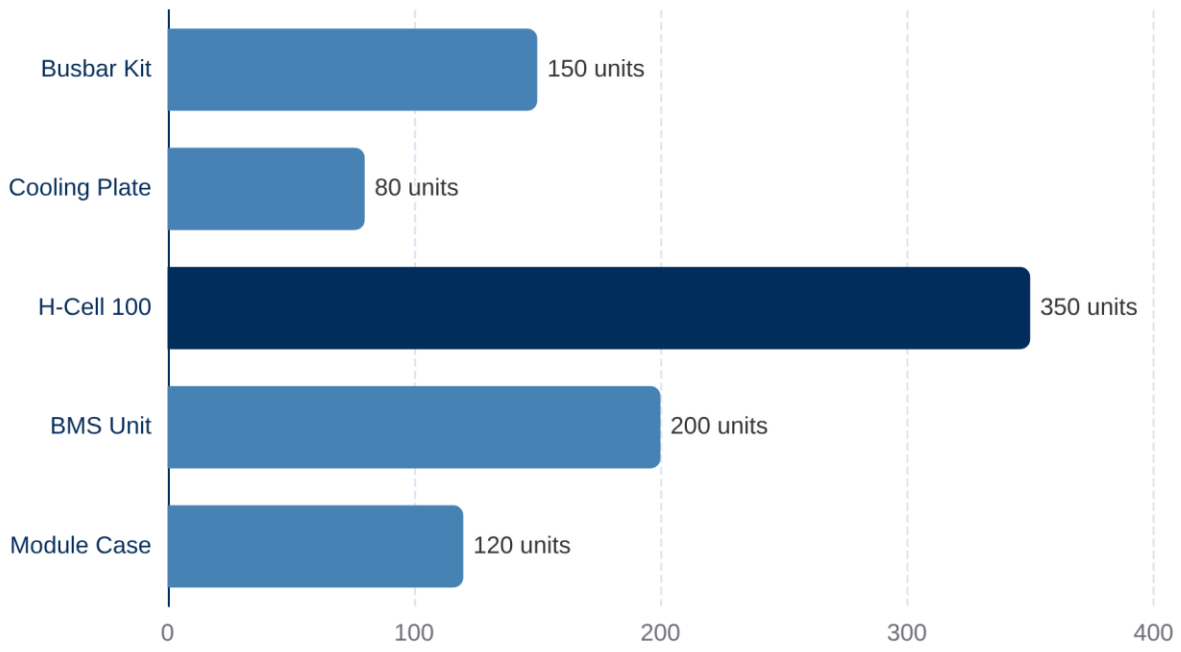
- 날짜 데이터 변환 시 옵션을 적용하여 비정상 형식 데이터를 결측치(NaT)로 일괄 변환
- 발생한 결측치는 를 통해 즉시 제거하여 AI 모델의 런타임 패닉(Panic) 및 오염 방지

데이터 그룹핑 및 가공 (Aggregation)

- Pandas 를 활용하여 일별 시계열 데이터를 분기(Quarter) 단위로 리샘플링
- 상품별 매출 합계 집계를 통해 '베스트 상품 Top 5'와 같은 경영 인사이트 데이터로 정규화 가공

가공 결과 예시: Best 5 Products (Quarterly)

전처리 후 도출된 분기별 핵심 상품 판매 비중 데이터



Data Integrity Verified Preprocessing Complete

[AI/데이터] Statsmodels 기반 Holt-Winters 시계열 매출 예측

자체 Python 환경에서 Statsmodels 라이브러리의 ExponentialSmoothing 알고리즘을 적용하여 외부 API 없이 정밀한 매출 예측을 수행합니다.

추세 및 계절성 동시 반영

단순 이동평균을 넘어 데이터의 장기적 추세(Trend)와 반복되는 주간 패턴(Seasonality)을 모두 학습하여 예측 정확도를 극대화함.

주간 반복 패턴 학습

설정을 통해 요일별 매출 변동 특성을 모델에 반영.

장기 예측 파이프라인

학습된 가중치를 바탕으로 향후 60일간의 매출액을 일별 단위로 정밀 예측하여 대시보드 및 RPA 리포트로 제공.

ALGORITHM CHARACTERISTICS

Trend Mode	Seasonality Mode
Additive ('add')	**Additive ('add')**

Seasonal Periods	Forecast Horizon
7 (Weekly)	**60 Days**

```
fromstatsmodels.tsa.holtwintersimportExponentialSmoothing

model = ExponentialSmoothing(
    data,
    trend='add',
    seasonal='add',
    seasonal_periods=7
)
fit = model.fit()
forecast = fit.forecast(60)
```

[Trouble Shooting] 데이터 부족 대응: 머신러닝 Fallback 아키텍처

AI/DATA PART

문제 상황 (Problem)

- 시스템 초기 도입 시나 신규 품목 등록 시 **데이터가 14일 미만**으로 존재할 경우 발생.
- Holt-Winters 모델 구동에 필요한 최소 계절성 주기 미달로 **Runtime Error** 및 서버 중단 위협.

해결책 (Solution)

try-except 예외 제어 구문을 통한 **하이브리드 모델링** 구현. 고도화 분석이 불가능한 시점에도 예측 서비스를 중단하지 않는 Fallback 로직 설계.

FALLBACK FLOW

Holt-Winters 시도 > 에러 감지 (< 14일) >

Linear Regression > Zero-Downtime

ANALYTICS_SERVICE.PY

```
try :
    # STEP 1. Holt-Winters (계절성 반영) 시도
        ExponentialSmoothing (
            train_data, trend='add', seasonal='add',
            seasonal_periods=7 ).fit().forecast(60)
        model = ExponentialSmoothing(
            train_data, trend='add', seasonal='add',
            seasonal_periods=7 ).fit()
        forecast = model.forecast(60)
        model_type = "Holt-Winters"

except Exception as e:
    # STEP 2. 데이터 부족 시 Linear Regression으로 Fallback
    print(f"Fallback Triggered: {e}")

    # Scikit-Learn 활용 단순 선형 회귀 모델 생성
        LinearRegression()
        model = LinearRegression()
        model.fit(X_train, y_train)
        forecast = model.predict(future_dates)
        model_type = "Linear Regression (Fallback)"

return forecast, model_type
```

데이터 안정성 확보 · 서비스 가용성 100%

[AI/데이터] 월 단위(ME) 리샘플링 장기 예측 및 웹 엑셀 연동

장기 계절성 예측 모델 구동 및 RPA 자동화를 위한 데이터 가공 파이프라인

장기 예측 모델링 전략

- 일별 데이터를 Pandas **resample('ME')**를 통해 월말 기준으로 압축하여 12개월 장기 예측 모델 구동
- Holt-Winters 알고리즘을 활용한 연간 계절성(Seasonality) 분석 및 미래 추세 예측

엑셀 빌드 및 RPA 연계

- 예측 결과 데이터를 Excel 파일 객체로 실시간 빌드하여 웹 프론트엔드 다운로드 링크 제공
- **RPA 타겟 소스:** 자동화 봇이 다운로드한 엑셀을 기반으로 경영진 메일 발송 프로세스 실행

DATA PIPELINE ARCHITECTURE

Raw Data
일별 원시 데이터
(DB 집계)

Resampling
Pandas
resample('ME')
월말 기준 압축

Forecasting
Holt-Winters
12개월 장기 예측

Excel Build
웹 다운로드
파일 생성

RPA Workflow
메일 자동 발송
(최종 타겟)

시스템 도입 성과 및 기대 효과 (ROI)

비용, 생산성, 무결성 측면의 정량적/정성적 가치 창출

외부 API 비용 Zero

보안성 확보 및 운영비 절감

- 고가 외부 클라우드 AI/LLM 의존도 0%
- 자체 온프레미스 모델 구축으로 보안 강화
- 지속적인 API 호출 비용(Running Cost) 전면 제거
- 기업 핵심 재무/생산 데이터 외부 유출 원천 차단

생산성 초격차

수기 작업 99% 단축

- 다단계 BOM 연쇄 차감 자동화 (0.1초 처리)
- Python OCR 기반 비정형 전표 인식 자동화
- Brity RPA 연동 경영진 리포트 메일 발송 무인화
- 재무/생산팀의 단순 반복 행정 업무 완전 폐기

무결성과 안정성

Zero-Downtime 시스템

- @Transactional 롤백으로 데이터 오염 방지
- AI 파이프라인 선형회귀 Fallback 우회 로직
- Append-Only 블랙박스 로그로 완벽한 추적성
- 예외 처리 UI/UX 파이프라인으로 사용자 혼선 방지

"무결점 엔터프라이즈 시스템 구축을 통한 제조 현장의 디지털 혁신 완성"

Retrospective

기술적 성장과 백엔드 엔지니어의 책임

엔지니어의 시각 변화

단순히 **"CRUD가 동작하는 기능 구현"**의 단계를 넘어, 실제 제조 현장의 복잡한 예외 상황으로부터 **"서버와 데이터를 어떻게 강력히 방어할 것인가"**를 깊이 체감했습니다.

무결성에 대한 집착

다단계 BOM의 재귀 호출과 트랜잭션 롤백 설계를 통해, 데이터의 1% 오차가 비즈니스 전체에 미치는 치명적인 영향을 이해하고 무결성을 위한 **방어적 프로그래밍** 습관을 확립했습니다.

순환 참조 방어

BOM 무한 루프를 차단하는 이중 검증 아키텍처 구축

원자성(Atomicity)

@Transactional을 통한 연쇄 차감 로직의 완벽한 롤백

수불 무결성

Append-Only 로그 설계를 통한 조작 불가능한 이력 관리

AI Fallback

데이터 부족 시 선형 회귀로 우회하는 Zero-Downtime 설계

The Power of a Single Line

"단 한 줄의 백엔드 트랜잭션 방어 코드가 공장 전체 라인의 셧다운을 막을 수 있음을 깨달았습니다."

Q&A

이상으로 발표를 마치겠습니다

감사합니다

ESS 배터리 통합 관리 시스템 및 온프레미스 AI/RPA 파이프라인

팀 총괄: 김강산

팀원: 정원준

팀원: 박주희

부록 (Reference & Tech Link)

프로젝트 상세 기술 문서 및 소스 코드 링크

GitHub Repository

전체 소스 코드, 커밋 히스토리 및 배포 스크립트가 포함된 원격 저장소입니다.

https://github.com/san123452/ESS_ERP.git

DB Schema & ERD

총 18개 테이블의 정규화 설계도 및 외래키(FK) 제약 조건 명세서입니다.

MariaDB 3NF Optimized

API Specifications

Spring Boot 컨트롤러와 Python FastAPI 간의 엔드포인트 통신 규약 문서입니다.

/docs/api-v5-spec.pdf

AI Analytics Script

```
import pandas as pd
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(data, trend='add', seasonal='add')
fit = model.fit()
forecast = fit.forecast(60)
```

test_analytics.py 핵심 로직 스니펫

ARCHITECTURE	SECURITY	DATA INTEGRITY	AUTOMATION
Polyglot MSA	On-Premise Only	Transactional Rollback	Brity RPA Workflow